COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010-CPE21S4	**SECTION:**	CPE21S4
DATE PERFORMED:	22/08/2026	**DATE SUBMITTED:**	22/08/2026
NAME:	Jullian Edrain R. Sevilla	**INSTRUCTOR:**	Engr. Jimlord Quejado

ACTIVITY NO. ___ [ACTIVITY NAME]

1. PROCEDURE OUTPUT

What procedures did you perform?

Welcome | Binary.cpp | LList.cpp | a.exe | Linear.cpp | LList.exe

6.1 > Binary.cpp > ...

```cpp
#include <iostream>
#include <cstdlib>
#include <ctime>
#include "searchingAlgo.h"

const int max_size = 50;

int main()
{

    //BINARY SEARCH
    std::cout << "\n\n===== BINARY SEARCH =====\n";

    // generate a sorted list for the array:
    int dataset2[max_size];
    for (int i = 0; i < max_size; i++)
    {
        dataset2[i] = i;
        std::cout << dataset2[i] << " ";
    }
    std::cout << "\n";

    binarySearchArr(11, dataset2, max_size);
    binarySearchArr(11111, dataset2, max_size);
}

```

Welcome | BInary.cpp | LList.cpp | a.exe | Linear.cpp | LList.exe

6.1 > Linear.cpp > main()

```cpp
#include <iostream>
#include <cstdlib>
#include <ctime>
#include "searchingAlgo.h"

const int max_size = 50;

int main()
{

    //LINEAR SEARCH

    std::cout << "===== LINEAR SEARCH =====\n";
    int dataset[max_size];
    srand(time(0));

    for (int i = 0; i < max_size; i++)
    {
        dataset[i] = rand() % 100;
        std::cout << dataset[i] << " ";
    }
    std::cout << "\n";

    int choice;
    std::cout << "Enter number to search: ";
    std::cin >> choice;
    linearSearchArr(max_size, choice, dataset);
}

```

Welcome | BInary.cpp | LList.cpp | a.exe | Linear.cpp | LList.exe

6.1 > LList.cpp > main()

```cpp
#include <iostream>
#include <cstdlib>
#include <ctime>
#include "searchingAlgo.h"

const int max_size = 50;

int main()
{

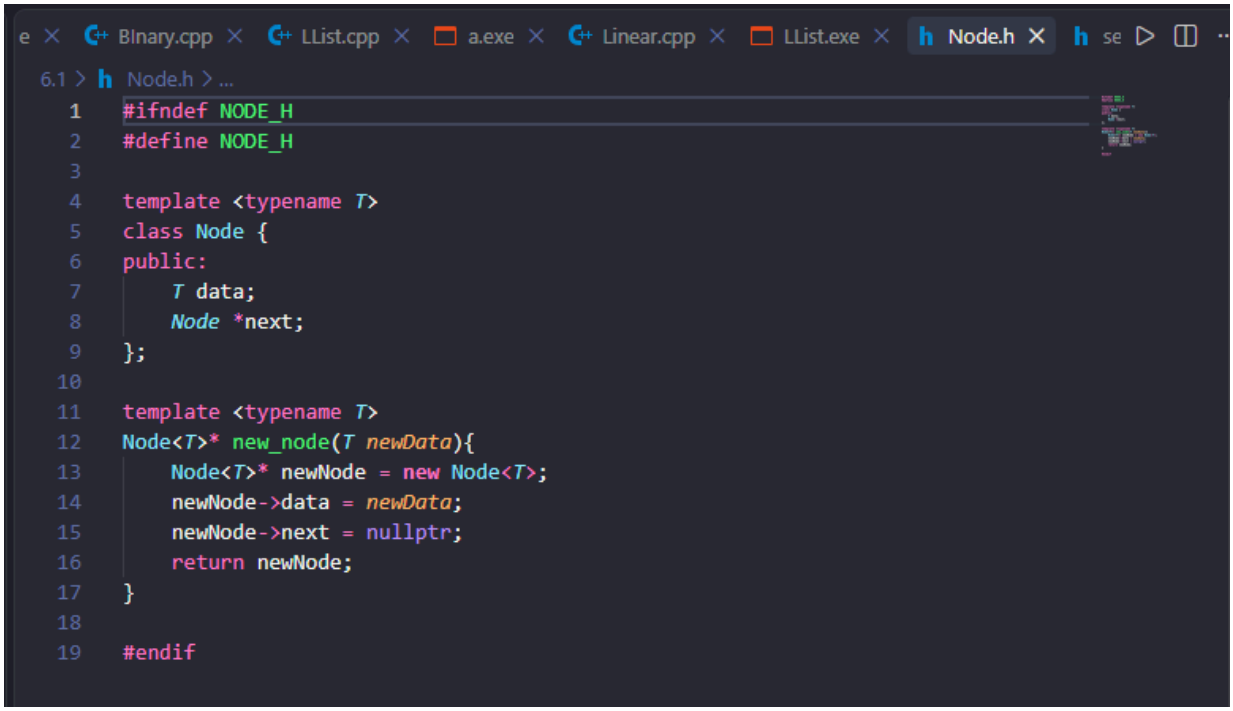
    //LINKED LIST "JULLIAN"
    std::cout << "\n===== LINKED LIST SEARCH =====\n";

    Node<char> *name1 = new_node('J');
    Node<char> *name2 = new_node('U');
    Node<char> *name3 = new_node('L');
    Node<char> *name4 = new_node('L');
    Node<char> *name5 = new_node('I');
    Node<char> *name6 = new_node('A');
    Node<char> *name7 = new_node('N');

    name1->next = name2;
    name2->next = name3;
    name3->next = name4;
    name4->next = name5;
    name5->next = name6;
    name6->next = name7;
    name7->next = nullptr;

    char letter;
    std::cout << "Enter letter to search in JULLIAN: ";
    std::cin >> letter;
    linearSearchLL(name1, letter);

    return 0;
}
```


e
BInary.cpp
LList.cpp
a.exe
Linear.cpp
LList.exe
Node.h
se
6.1 > Node.h > ...
#ifndef NODE_H
#define NODE_H
template <typename T>
class Node {
public:
T data;
Node *next;
};
template <typename T>
Node<T>* new_node(T newData){
Node<T>* newNode = new Node<T>;
newNode->data = newData;
newNode->next = nullptr;
return newNode;
}
#endif

Welcome | Binary.cpp | LList.cpp | a.exe | Linear.cpp | LList.exe | Node.h | searchingAlgo.h

6.1 > searchingAlgo.h > ...

```cpp
#ifndef SEARCHINGALGO_H
#define SEARCHINGALGO_H

#include <iostream>

// NODE STRUCT & HELPER
template <typename T>
struct Node
{
    T data;
    Node *next;
};

template <typename T>
Node<T> *new_node(T value)
{
    Node<T> *temp = new Node<T>;
    temp->data = value;
    temp->next = nullptr;
    return temp;
}

// LINEAR SEARCH – ARRAY
template <typename T>
void linearSearchArr(int maxSize, T dataFind, T data[])
{
    int i = 0;
    while (i < maxSize)
    {
        if (dataFind == data[i])
        {
            std::cout << "Searching is successful!\n";
            return;
        }
        i++;
    }
    std::cout << "Searching is unsuccessful!\n";
}

// BINARY SEARCH – ARRAY (YOUR VERSION)
template <typename T>
void binarySearchArr(T dataFind, T data[], int maxSize)
{
    int low = 0, up = maxSize - 1;
    int middle;

    while (low <= up)
    {
```

Welcome | Binary.cpp | LList.cpp | a.exe | Linear.cpp | LList.exe | Node.h | searchingAlgo.h

6.1 > searchingAlgo.h > ...

```
#ifndef SEARCHINGALGO_H
void binarySearchArr(T dataFind, T data[], int maxSize)
    while (low <= up)
    {
        middle = (low + up) / 2;

        if (dataFind == data[middle])
        {
            std::cout << "Searching is successful!\n";
            return;
        }
        else if (dataFind < data[middle])
        {
            up = middle - 1;
        }
        else
        {
            low = middle + 1;
        }
    }
    std::cout << "Searching is unsuccessful!\n";
}

// LINEAR SEARCH – LINKED LIST
template <typename T>
void linearSearchLL(Node<T> *node, T dataFind)
{
    while (node != nullptr)
    {
        if (node->data == dataFind)
        {
            std::cout << "Searching is successful!\n";
            return;
        }
        node = node->next;
    }
    std::cout << "Searching is unsuccessful!\n";
}

// CREATE A HELPER FUNCTION TO GET THE MIDDLE
template <typename T>
Node<T> *getMiddle(Node<T> *start, Node<T> *last)
{

    // use two pointers
    // slow moves one node at a time
    // fast moves two at a time
    Node<T> *slow = start;
```

Welcome | Binary.cpp | LList.cpp | a.exe | Linear.cpp | LList.exe | Node.h | searchingAlgo.h

6.1 > searchingAlgo.h > getMiddle<T>(Node<T>*, Node<T>*)

```
1   #ifndef SEARCHINGALGO_H
86  Node<T> *getMiddle(Node<T> *start, Node<T> *last)
92      Node<T> *slow = start;
93      Node<T> *fast = start;
94
95      // create a loop until fast reaches the end or
96      // reaches the last node of the current search range
97      while (fast != last && fast->next != last)
98      {
99
100         slow = slow->next;
101         fast = fast->next->next;
102     }
103
104     return slow;
105 }
106
107 template <typename T>
108 void binarySearchLL(Node<T> *start, T dataFind)
109 {
110
111     Node<T> *last = nullptr;
112
113     while (start != last)
114     {
115
116         // middle element
117         Node<T> *middle = getMiddle(start, last);
118
119         if (middle->data == dataFind)
120         {
121             std::cout << "the data was found. \n";
122             return;
123         }
124         else if (middle->data > dataFind)
125         {
126             last = middle;
127         }
128         else
129         {
130             start = middle->next;
131         }
132     }
133
134     std::cout << "Search element was not found\n";
135 }
136
137 #endif
```

```
Suggestion [3,General]: The command BInary.cpp was not found, but does exist in the current location. Windows PowerShell does not load commands from the current location by default. If you trust this command, instead t
ype: ".\BInary.cpp". See "get-help about_Command_Precedence" for more details.
PS C:\Users\Jullian Sevilla\OneDrive\Documents\DSA\Midterms\6.1> g++ BInary.cpp -o output
PS C:\Users\Jullian Sevilla\OneDrive\Documents\DSA\Midterms\6.1> ./output
>>


===== BINARY SEARCH =====
0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31 32 33 34 35 36 37 38 39 40 41 42 43 44 45 46 47 48 49
Searching is successful!
Searching is unsuccessful!
PS C:\Users\Jullian Sevilla\OneDrive\Documents\DSA\Midterms\6.1> g++ Linear.cpp -o output
PS C:\Users\Jullian Sevilla\OneDrive\Documents\DSA\Midterms\6.1> ./output
===== LINEAR SEARCH =====
92 22 93 56 95 81 23 4 59 49 32 62 96 46 21 74 40 38 93 86 75 0 10 75 22 7 33 85 95 41 38 81 84 82 37 89 14 47 16 84 79 39 21 47 87 0 92 51 68 89
Enter number to search: 100
Searching is unsuccessful!
PS C:\Users\Jullian Sevilla\OneDrive\Documents\DSA\Midterms\6.1> g++ LList.cpp -o output
PS C:\Users\Jullian Sevilla\OneDrive\Documents\DSA\Midterms\6.1> ./output

===== LINKED LIST SEARCH =====
Enter letter to search in JULLIAN: J
Searching is successful!
PS C:\Users\Jullian Sevilla\OneDrive\Documents\DSA\Midterms\6.1>
```

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Copy and paste the questions from the lab and include your answers after.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: _____

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: _____

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: _____

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.

7. Supplementary Activity

ILO B: Solve different problems utilizing appropriate searching techniques in C++

For each provided problem, give a screenshot of your code, the output console, and your answers to the questions.

Problem 1. Suppose you are doing a sequential search of the list [15, 18, 2, 19, 18, 0, 8, 14, 19, 14]. Utilizing both a linked list and an array approach to the list, use sequential search and identify how many comparisons would be necessary to find the key '18'?

Problem 2. Modify your sequential search algorithm so that it returns the count of repeating instances for a given search element 'k'. Test on the same list given in problem 1.

Problem 3. Suppose you have the following sorted list [3, 5, 6, 8, 11, 12, 14, 15, 17, 18] and are using the binary search algorithm. If you wanted to find the key 8, draw a diagram that shows how the searching works per iteration of the algorithm. Prove that your drawing is correct by implementing the algorithm and showing a screenshot of the code and the output console.

Problem 4. Modify the binary search algorithm so that the algorithm becomes recursive. Using this new recursive binary search, implement a solution to the same problem for problem 3.